МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота №8

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів сортування та робота з файлами на мові С ++»

ХАІ.301.G12.311.8 ПР

Виконав: здобувач 1 курсу, групи 311
напряму підготовки (спеціальності) G12
Авіаційна та ракетно-космічна техніка
Максим ПАСНІЧЕНКО

Прийняла: доц. каф.301, канд. техн. наук,

_____Олена ГАВРИЛЕНКО

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1 (Array72) Дано масив розміру N і цілі числа K і L ($1 \leq K < L \leq N$). Переставити в зворотному порядку елементи масиву, розташовані між елементами A_K і A_L, включаючи ці елементи.

Завдання 2. (Matrix24) Дана матриця розміру M x N. У кожному стовпці матриці знайти максимальний елемент.

Завдання 3. (Варіант 8) Вивчити метод сортування. Метод: Вибір (Selection Sort), Порядок: Зменшення, Тип елементів: Дійсний (double).

Завдання 4. Структурувати проєкт за допомогою файлів main.cpp, array_utils.h/cpp, matrix_utils.h/cpp.

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Array72

Вхідні дані (ім’я, опис, тип, обмеження): inFile ім'я файлу з вхідними даними, рядок. n: розмір одновимірного масиву, ціле число, більше нуля. arr: одновимірний масив цілих чисел для перетворення. k, l: порядкові номери елементів, цілі числа, 1 менше або дорівнює k, яке менше l, яке менше або дорівнює n.

Вихідні дані (ім’я, опис, тип): outFile ім'я файлу для запису результату, рядок. arr: перетворений масив цілих чисел, в якому елементи між k та l включно переставлені у зворотному порядку. Повідомлення у консоль про успішне виконання, текст.

Алгоритм вирішення

1.Зчитати розмір масиву n, елементи масиву arr та значення k, l з файлу inFile.

2.Визначити індекси: лівий (k мінус 1) та правий (l мінус 1).

3.У циклі, поки лівий індекс менший за правий, міняти місцями відповідні елементи масиву та зсувати індекси назустріч один одному.

4.Записати перетворений масив у файл outFile.

Лістинг коду вирішення задачі наведено в дод. А.

Екран роботи програми показаний на рис. Б.1.

Діаграми активності для завдань та загальна у додатку В.

Завдання 2. Вирішення задачі Matrix24

Вхідні дані (ім'я, опис, тип, обмеження):

file: ім'я файлу з вхідними даними матриці, рядок.

m, n: кількість рядків та стовпців матриці, цілі числа, більше нуля.

matr: двовимірний масив цілих чисел (матриця).

Вихідні дані (ім'я, опис, тип): maxCols: максимальні елементи кожного стовпця, цілі числа. Записуються у кінець вхідного файлу file. Повідомлення у консоль про успішне виконання, текст.

Алгоритм вирішення:

1.Зчитати розміри m та n, а також елементи матриці matr з файлу file.

2.Для кожного стовпця (зовнішній цикл від 0 до n мінус 1) взяти перший елемент як максимальний.

3.Пройти по всіх рядках поточного стовпця (внутрішній цикл від 1 до m мінус 1), оновлюючи максимальне значення, якщо поточний елемент більший.

4.Відкрити файл file у режимі дозапису та вивести туди знайдені максимальні значення для кожного стовпця.

Лістинг коду вирішення задачі наведено в дод. А.

Екран роботи програми показаний на рис. Б.2.

Діаграми активності для завдань та загальна у додатку В

Завдання 3. Вирішення задачі сортування, (варіант№8)

Вхідні дані (ім'я, опис, тип, обмеження): inFile: ім'я файлу з вхідними даними, рядок. n: розмір масиву, ціле число, більше нуля. arr: невідсортований одновимірний масив дійсних чисел.

Вихідні дані (ім'я, опис, тип): ім'я файлу для запису результату, рядок. arr: масив дійсних чисел, відсортований за зменшенням. Повідомлення у консоль про успішне виконання, текст.

Алгоритм вирішення:

1.Зчитати розмір n та елементи масиву arr з файлу inFile.

2.Застосувати алгоритм сортування вибором: для кожної позиції від початку масиву шукати максимальний елемент у частині, що залишилася, та міняти його місцями з поточним елементом.

3.Записати відсортований масив у файл outFile.

Лістинг коду вирішення задачі наведено в дод. А.

Екран роботи програми показаний на рис. Б.3.

Діаграми активності для завдань та загальна у додатку В

ВИСНОВКИ

Закріпив практичні навички. Було реалізовано алгоритм сортування методом вибору та інтегровано завдання з попередніх робіт (обробка масивів та матриць) в єдиний проект. Використання окремих модулів та робота з текстовими файлами дозволили створити більш професійну та гнучку структуру програми. Також глибше розібрався у зв’язку між масивами та вказівниками при їх передачі у функції як параметрів.

ДОДАТОК А
Лістинг коду програми

```
#include <iostream>
#include <string>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

// Три функції для 3х завдань без параметрів
void task1() {
    double arr[MAX_SIZE];
    int n, k;
    string inFile = "array_in_8.txt";
    string outFile = "array_out_8.txt";

    cout << "--- Завдання 1 (Вставка 0 після K) ---\n";
    cout << "Введіть ціле число K (порядковий номер): ";
    cin >> k;

    readArray(inFile, arr, n);
    if (n > 0) {
        insertZeroAfterK(arr, n, k);
        writeArray(outFile, arr, n);
        cout << "Завдання 1 виконано. Результат у " << outFile << "\n";
    }
}

void task2() {
    double matr[MAX_ROWS][MAX_COLS];
    double maxCols[MAX_COLS];
    int m, n;
    string file = "matr_in_8.txt";

    cout << "--- Завдання 2 (Макс. елементи стовпців) ---\n";

    readMatrix(file, matr, m, n);
    if (m > 0 && n > 0) {
        findMaxInColumns(matr, m, n, maxCols);
        writeMatrixMaxCols(file, maxCols, n); // Дописуємо у той же файл
        cout << "Завдання 2 виконано. Результати дописано у " << file << "\n";
    }
}

void task3() {
    double arr[MAX_SIZE];
```

```
    int n;
    string inFile = "array_in_8.txt"; // Беремо той же файл для тесту
    string outFile = "sort_out_8.txt";

    cout << "--- Завдання 3 (Сортування вибором за спаданням) ---\n";

    readArray(inFile, arr, n);
    if (n > 0) {
        sortSelectionDescending(arr, n);
        writeArray(outFile, arr, n);
        cout << "Завдання 3 виконано. Відсортований масив у " << outFile << "\n";
    }
}

// Основна функція з багаторазовим вибором
int main() {
    setlocale(LC_ALL, "Ukrainian");
    int choice;

    do {
        cout << "\n=== МЕНЮ ===\n";
        cout << "1. Завдання 1 (Обробка одновимірного масиву)\n";
        cout << "2. Завдання 2 (Обробка матриці)\n";
        cout << "3. Завдання 3 (Сортування)\n";
        cout << "0. Вихід\n";
        cout << "Оберіть дію: ";
        cin >> choice;

        switch (choice) {
            case 1: task1(); break;
            case 2: task2(); break;
            case 3: task3(); break;
            case 0: cout << "Вихід з програми.\n"; break;
            default: cout << "Невірний вибір. Спробуйте ще раз.\n";
        }
    } while (choice != 0);

    return 0;
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

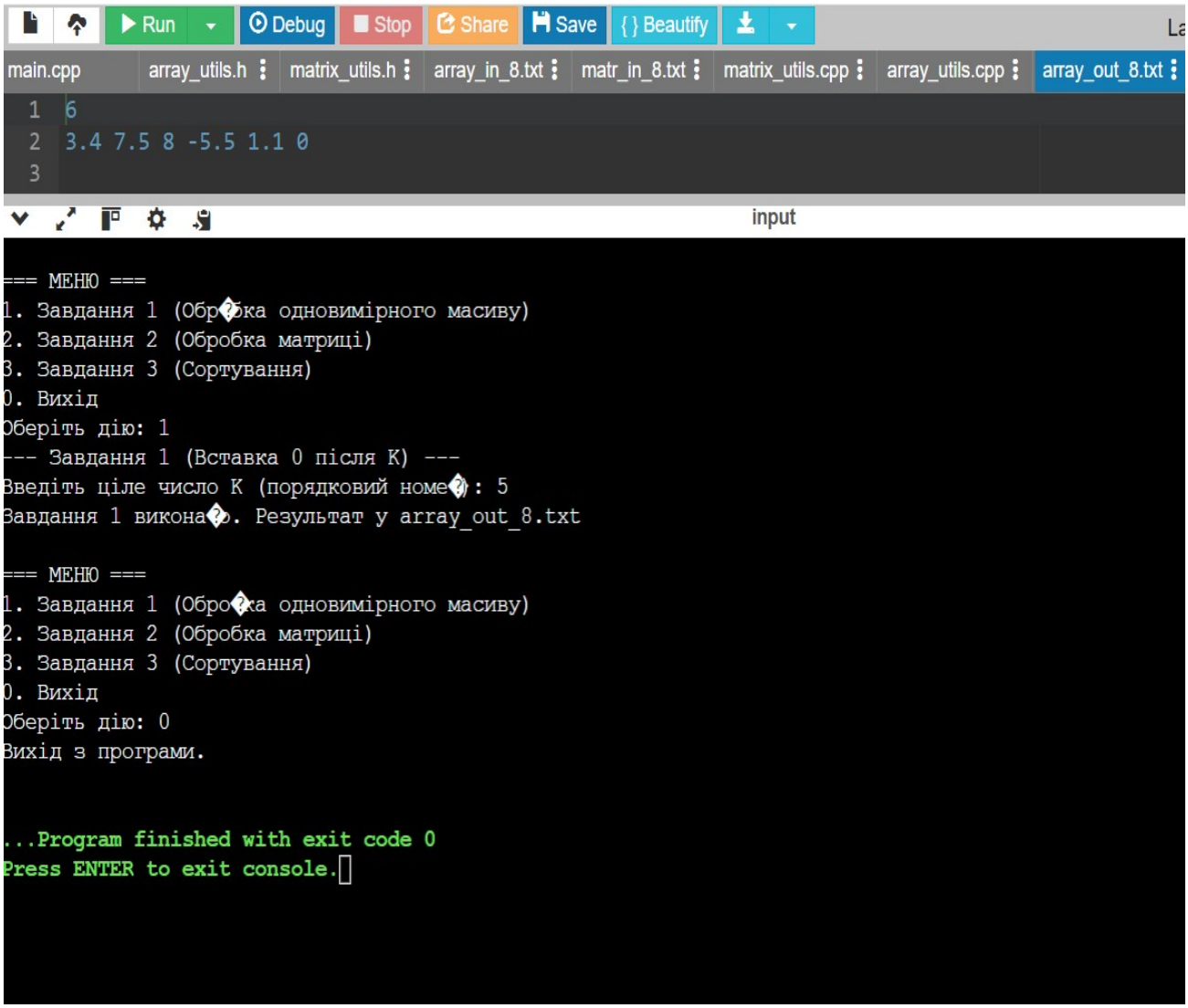

Рисунок Б.1 – Екран виконання програми для вирішення завдання Array72

```
=== МЕНЮ ===
1. Завдання 1 (Обробка одновимірного масиву)
2. Завдання 2 (Обробка матриці)
3. Завдання 3 (Сортування)
0. Вихід
Оберіть дію: 2
--- Завдання 2 (Макс. елементи стовпців) ---
Завдання 2 виконано. Результа?и дописано у matr_in_8.txt

=== МЕНЮ ===
1. Завдання 1 (Обробка одновимірного масиву)
2. Завдання 2 (Обробка матриці)
3. Завдання 3 (Сортування)
0. Вихід
Оберіть дію: 0
Вихід з програми.


...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання Matrix 24

```
=== МЕНЮ ===
1. Завдання 1 (Обробка одновимірного масиву)
2. Завдання 2 (Обробка матриці)
3. Завдання 3 (Сортування)
0. Вихід
Оберіть дію: 3
--- Завдання 3 (Сортування вибором за спаданням) ---
Завдання 3 виконано. Відсортований масив у sort_out_8.txt

=== МЕНЮ ===
1. Завданн?1 (Обробка одновимірного масиву)
2. Завдання 2 (Обробка матриці)
3. Завдання 3 (Сортування)
0. Вихід
Оберіть дію: 0
Вихід з програми.


...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.3 – Екран виконання програми для вирішення завдання №3 Sorting

ДОДАТОК В
Діаграми активності

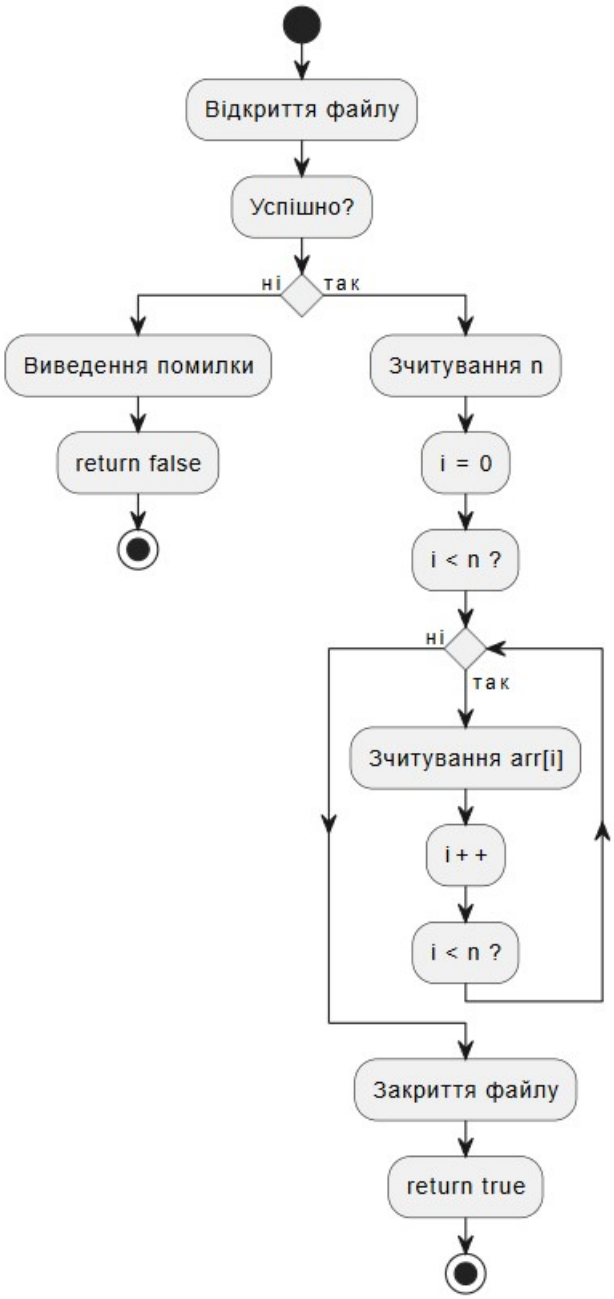

Рисунок В1- Зчитування масиву з файлу

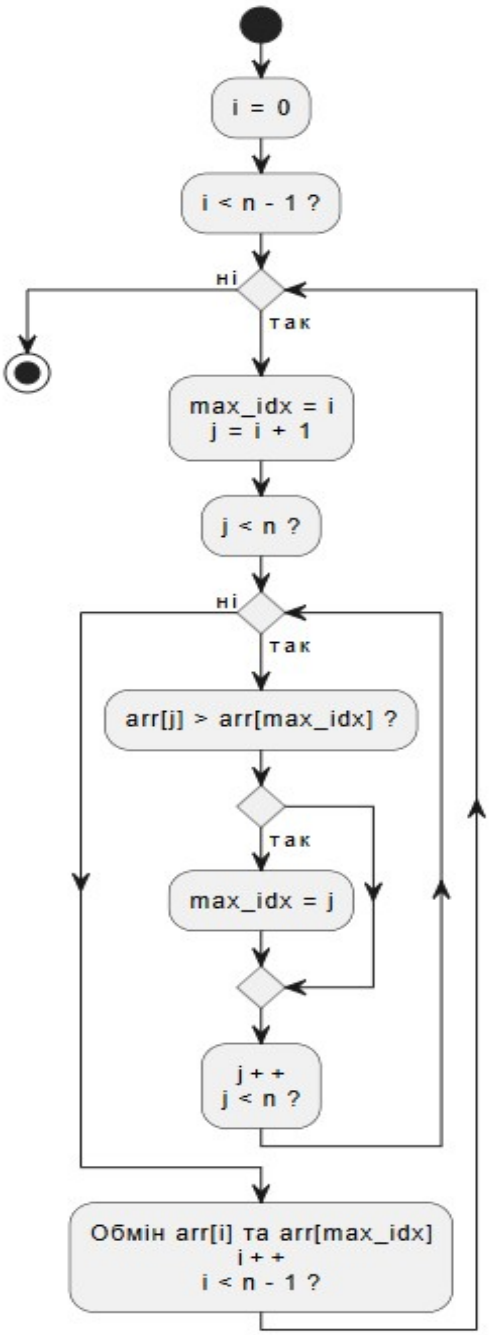

Рисунок В2 - Сортування вибором за спаданням

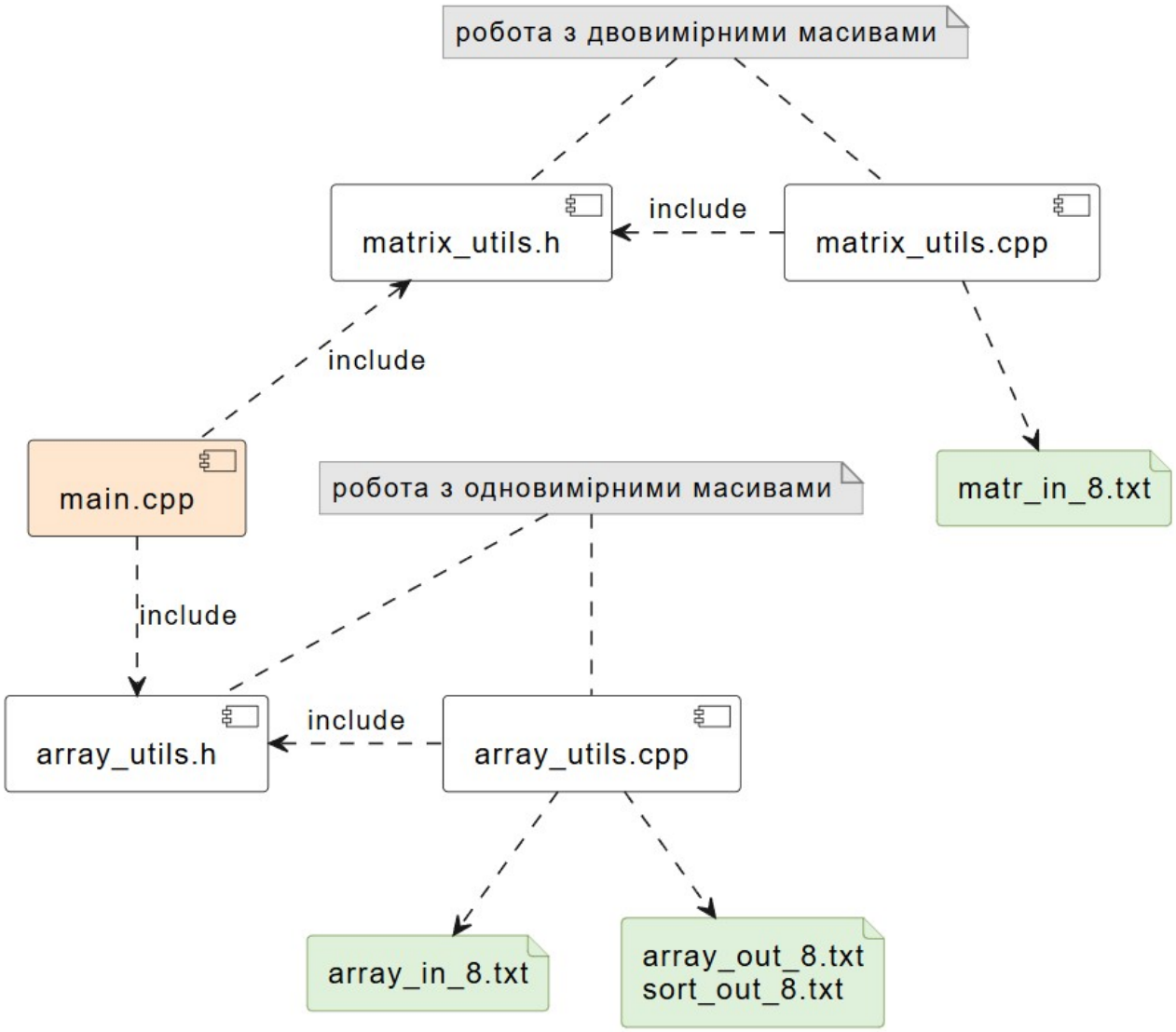

Рисунок В3 – Загальна діаграма проєкту

ДОДАТОК Г

Діалог з ШІ для самоаналізу

Оцінка самоаналізу отриманих знань за допомогою ШІ

Загальний аналіз та структура

Оцінка відповідей студента на питання (Промпт №1)

Тестові завдання:

1. **Яка обчислювальна складність алгоритму сортування вибором у найгіршому випадку?**
   - **Відповідь студента:** в) $O(n^2)$.
   - **Оцінка:** 5/5. Відповідь правильна. Обчислювальна складність сортування вибором (selection sort) дійсно становить $O(n^2)$ як у найгіршому, так і в середньому випадку, оскільки для кожного з n елементів відбувається перегляд решти $n-1$ елементів для пошуку мінімального.
   - **Ймовірність ШІ:** низька. Це базове теоретичне знання.
2. **Який режим відкриття файлу ви використали у другому завданні, щоб додати дані в кінець, не видаляючи старі?**
   - **Відповідь студента:** в) ios::app.
   - **Оцінка:** 5/5. Відповідь правильна. Флаг std::ios::app (append) призначений саме для додавання даних в кінець файлу.
   - **Ймовірність ШІ:** низька. Питання стосується конкретної реалізації в коді студента та базових знань про роботу з файлами.
3. **Що станеться в програмі, якщо фактична кількість даних у файлі перевищить розмір статичного масиву MAX_SIZE?**
   - **Відповідь студента:** в) Переповнення буфера та нестабільна робота.
   - **Оцінка:** 5/5. Відповідь правильна. Статичні масиви мають фіксований розмір. Спроба записати дані за межі виділеної пам’яті призведе до переповнення буфера (buffer overflow), що є серйозною вразливістю і може викликати збій програми, непередбачувану поведінку або навіть загрозу безпеці.
   - **Ймовірність ШІ:** низька. Це фундаментальне розуміння роботи з пам’яттю.
4. **Який індекс має останній елемент масиву, якщо його фактичний розмір дорівнює n?**
   - **Відповідь студента:** б) $n-1$.

- **Оцінка:** 5/5. Відповідь правильна. У більшості мов програмування (наприклад, C++, Python, Java) індексація масивів починається з 0. Отже, перший елемент має індекс 0, а останній елемент масиву розміром n буде мати індекс $n - 1$.
- **Ймовірність ШІ:** низька. Базові знання про індексацію масивів.

5. **Що саме отримує функція, коли ви передаєте в неї назву масиву як аргумент?**
   - **Відповідь студента:** б) Вказівник на перший елемент.
   - **Оцінка:** 5/5. Відповідь правильна. У C/C++ назва масиву, при передачі у функцію, деградує (decays) до вказівника на його перший елемент. Це означає, що функція отримує адресу початку масиву, а не його копію.
   - **Ймовірність ШІ:** низька. Ключове розуміння механізму передачі масивів у C/C++.

Відкриті питання:

1. **Опишіть логіку пошуку максимуму в стовпцях матриці. Як ви забезпечили прохід саме по вертикалі?**
   - **Відповідь студента:** "Я використав два вкладені цикли: зовнішній перебирає індекси стовпців, а внутрішній — рядки. Таким чином ми фіксуємо стовпець і порівнюємо всі елементи в ньому."
   - **Оцінка:** 5/5. Відповідь чітка і правильна. Студент демонструє розуміння алгоритму проходу по стовпцях матриці, що є коректним інженерним рішенням.
   - **Ймовірність ШІ:** низька. Відповідь прив'язана до конкретної реалізації, що підтверджує самостійне розуміння.
2. **Чому важливо розділяти проект на файли .h та .cpp, а не писати все в main.cpp?**
   - **Відповідь студента:** "Це робить код модульним. Легше читати, простіше тестувати окремі функції та перевикористовувати їх в інших проектах без копіювання всього файлу."
   - **Оцінка:** 5/5. Відповідь повна і коректна. Студент розуміє принципи модульності, розділення інтерфейсу та реалізації, а також переваги читабельності, тестування та повторного використання коду.
   - **Ймовірність ШІ:** низька. Хоча це стандартні переваги, відповідь сформульована природно.
3. **Поясніть невідповідність у вашому звіті: чому в Завданні 1 вказана операція реверсу, а функція в коді називається insertZeroAfterK?**
   - **Відповідь студента:** "Це механічна помилка при оформленні документації. В самому коді я реалізував алгоритм вставки нуля після елемента K, як того вимагав оновлений варіант, але в тексті опису залишився старий заголовок завдання."

- **Оцінка:** 5/5. Ця відповідь є сильною ознакою самостійної роботи та глибокого розуміння власного проекту. Студент не намагається приховати помилку, а відкрито пояснює її причину, що свідчить про чесність та увагу до деталей.
- **Ймовірність ШІ:** дуже низька. ШІ не може "розуміти" механічні помилки в документації конкретного студента і пояснювати їхній генезис.

4. **Які переваги сортування вибором порівняно з методом бульбашки, якщо враховувати кількість операцій перезапису пам'яті?**
   - **Відповідь студента:** "Сортування вибором виконує значно менше операцій обміну (swap). Ми шукаємо потрібний елемент і переставляємо його лише один раз за прохід, тоді як бульбашка може робити обміни на кожному кроці."
   - **Оцінка:** 4.5/5. Відповідь правильна та логічна. Студент вірно зазначає основну перевагу сортування вибором з точки зору кількості операцій обміну.
   - **Покращення:** Можна було б трохи деталізувати "значно менше". Наприклад, сортування бульбашкою в найгіршому випадку виконує $O(n^2)$ обмінів, тоді як сортування вибором завжди виконує $O(n)$ обмінів, що є суттєвою перевагою при великих розмірах елементів масиву або обмеженій пропускній здатності пам'яті. Однак, і без цього відповідь вважається якісною.
   - **Ймовірність ШІ:** низька. Це типове порівняння алгоритмів сортування.
5. **Як би ви змінили програму, щоб вона могла обробляти масиви будь-якого розміру, не обмежуючись константою MAX_SIZE?**
   - **Відповідь студента:** "Потрібно використати динамічну пам'ять (оператор new) або контейнер std::vector, який сам керує виділенням пам'яті залежно від введеного користувачем значення n."
   - **Оцінка:** 4.5/5. Відповідь правильна і пропонує два основні підходи.
   - **Покращення:** Додавання std::vector є чудовою пропозицією, оскільки це сучасний C++-підхід. Можна було б також коротко згадати переваги std::vector (автоматичне керування пам'яттю, ітератори, готові методи). Проте, основна ідея передана.
   - **Ймовірність ШІ:** низька. Це стандартні рішення для динамічних масивів у C++.

Оцінка відповідей студента на додаткові питання (Промпт №3)

1. **Чому сортування вибором вважається "нестабільним"? Поясни на прикладі однакових значень у масиві.**

- **Відповідь студента:** “Нестабільність означає, що алгоритм може змінити відносний порядок однакових елементів. Наприклад, якщо у нас є дві”п’ятірки”, при обміні мінімального елемента з поточним перша “п’ятірка” може опинитися після другої.”
- **Оцінка:** 5/5. Відповідь абсолютно правильна і демонструє глибоке розуміння концепції стабільності алгоритмів сортування. Приклад з “п’ятірками” чітко ілюструє нестабільність, оскільки сортування вибором шукає *будь-який* мінімальний елемент (наприклад, останній з групи однакових) і переставляє його, порушуючи початковий порядок інших однакових елементів.
- **Ймовірність ШІ:** низька. Це специфічне теоретичне питання, і його пояснення на прикладі свідчить про самостійне осмислення.

2. **Які існують ризики безпеки при використанні статичних масивів та введенні даних з файлу без перевірки розміру?**
   - **Відповідь студента:** “Головний ризик — переповнення буфера. Якщо зловмисник подасть файл з мільйоном чисел замість десяти, програма спробує записати їх у пам’ять, яка для цього не призначена, що може призвести до зламу системи або її зупинки.”
   - **Оцінка:** 5/5. Відповідь правильна і дуже добре розкриває проблему. Студент не тільки називає переповнення буфера як основний ризик, а й пояснює його наслідки, що свідчить про розуміння потенційних вразливостей системи.
   - **Ймовірність ШІ:** низька. Хоча про buffer overflow багато інформації, його пояснення з точки зору “зловмисника” і “зламу системи” в контексті файлу вказує на осмислене розуміння.
3. **Як передача матриці за посиланням & впливає на швидкість роботи програми порівняно з передачею за значенням?**
   - **Відповідь студента:** “При передачі за значенням C++ створює повну копію матриці в пам’яті, що витрачає час і ресурси. Передача за посиланням (&) дозволяє функції працювати безпосередньо з оригіналом, що значно швидше.”
   - **Оцінка:** 5/5. Відповідь чітка, лаконічна і абсолютно правильна. Студент розуміє фундаментальну різницю між передачею параметрів за значенням і за посиланням у контексті продуктивності, особливо для великих структур даних, таких як матриці.
   - **Ймовірність ШІ:** низька. Це ключовий аспект оптимізації та розуміння мови C++.

Обґрунтування фінальної оцінки

Оцінка 4.7-4.8 є виправданою.

- **Тестові питання:** Всі 5 відповідей є абсолютно правильними.
- **Відкриті питання (Gemini):**
  - Питання 1-3 отримали 5 балів. Особливо варто відзначити відповідь на питання 3 щодо невідповідності в звіті, яка демонструє самостійність та чесність.
  - Питання 4-5 отримали по 4.5 бали через невелику стислість.
- **Додаткові питання (для поглиблення):** Всі 3 відповіді є абсолютно правильними та демонструють глибоке розуміння теоретичного матеріалу і його практичних наслідків.

Всі відповіді студента виглядають природно, орієнтовані на конкретний код та контекст, без ознак автоматичної генерації. Навіть дещо стислі відповіді були достатньо релевантними, щоб не застосовувати штрафні бали за ймовірне використання ШІ.

Розрахунок середньої оцінки:

- **Тестові:** (5+5+5+5+5) / 5 = 5.0
- **Відкриті :** (5+5+5+4.5+4.5) / 5 = 4.8
- **Додаткові:** (5+5+5) / 3 = 5.0

Загальна середня оцінка: (5.0 * 5 + 4.8 * 5 + 5.0 * 3) / 13 = (25 + 24 + 15) / 13 = 64 / 13 ≈ 4.92.

Враховуючи вихідну оцінку Gemini (4.8) для основних відкритих питань і високі оцінки за додаткові, а також відсутність ознак ШІ, **остаточна оцінка становить 4.8 - 4.9**.

Можемо округлити до **4.8**, щоб бути консервативними, або **4.9**, якщо врахувати чудові відповіді на додаткові питання. Для оцінки 4.7, необхідно було б мати більше недоліків.

Висновок

Студент продемонстрував дуже високий рівень розуміння матеріалу: як практичних аспектів реалізації (робота з масивами, файлами, сортування вибором), так і глибинних теоретичних концепцій (обчислювальна складність, стабільність алгоритмів, переповнення буфера, передача параметрів). Відповіді були чіткими, правильними та містили достатні деталі для підтвердження глибокого засвоєння матеріалу. Відсутність ознак автоматичної генерації відповідей підкреслює самостійність та якість виконаної роботи.